

POLITECHNIKA RZESZOWSKA
KATEDRA INFORMATYKI I AUTOMATYKI
DR INŻ. MATEUSZ POMIANEK

ĆWICZENIE LABORATORYJNE FLUTTER 2

Riverpod, animacje, Hive i testy widgetów

Programowanie Aplikacji Mobilnych - moduł Fluttera

Projekt: WeatherApp v2 - rozszerzenie projektu z Flutter Lab 1

Wymagania wstępne: ukończony Flutter Lab 1 (Dart, widgety, Provider, GoRouter, http)

Spis treści

1. Od Provider do Riverpod - dlaczego warto przejść?	1
2. Typy providerów - kiedy którego użyć?	3
3. Hive - lokalna baza danych NoSQL	6
4. Animacje niejawne - Animated* widgety	9
5. Animacje jawne - AnimationController	12
6. Testy widgetów - flutter_test	14
7. Mocktail - mockowanie zależności	18
8. Architektura projektu - Feature-First i Clean Architecture	19
9. Dostępność i internacjonalizacja	21
10. Flutter DevTools - diagnostyka i profilowanie	23
11. Zadania do wykonania	24
12. Kryteria oceniania	25
13. Najczęstsze błędy i ich rozwiązania	26

1. Od Provider do Riverpod - dlaczego warto przejść?

W Flutter Lab 1 zarządzaliśmy stanem przy pomocy Provider i ChangeNotifier. Provider jest świetny na start, ale w miarę rozrostu aplikacji ujawnia ograniczenia: trudność z zależnościami między providerami, brak wsparcia dla asynchroniczności, konieczność ręcznego dispose(). Riverpod

rozwiązuje te problemy i jest oficjalnie rekomendowanym następcą Providera przez tego samego autora - Rémi Rousselet.

1.1. Problemy z Provider w dużych aplikacjach

Problem z Provider	Jak Riverpod to rozwiązuje
Zależności między providerami wymagają uważnej kolejności w drzewie widgetów. Błędna kolejność = brak dostępu.	Providery Riverpoda są globalne i mogą odczytywać siebie nawzajem bez zależności od drzewa widgetów.
ChangeNotifier musi być ręcznie dispose()owany. Zapomnienie to wyciek pamięci.	Riverpod automatycznie zarządza cyklem życia - niszczy providera gdy nikt go nie obserwuje.
Brak natywnego wsparcia dla Future/Stream w jednym providerze. Trzeba samemu obsługiwać loading/error.	AsyncNotifierProvider i FutureProvider wbudowują obsługę trzech stanów: loading, data, error (AsyncValue).
Context.read() w callbackach może rzucić wyjątek jeśli widget jest nieaktywny.	ref.read() jest bezpieczne niezależnie od cyklu życia widgetu.
Trudne testowanie - trzeba owijać widgety w ProviderScope i dbać o kolejność.	Testy Riverpoda są proste: nadpisz jeden provider przez ProviderScope(overrides:[]).

Analogia: tablica ogłoszeń vs. radio na żywo

[PROVIDER] Tablica ogłoszeń w korytarzu szkoły: ogłoszenia wiszą w konkretnym miejscu. Możesz je przeczytać tylko jeśli jesteś w tym korytarzu (kontekst!). Jeśli zmienisz korytarz (przejdiesz do innego ekranu), musisz szukać innej tablicy.

[RIVERPOD] Radio na żywo: możesz dostroić się do dowolnej stacji z dowolnego miejsca w domu. Nie musisz wiedzieć gdzie jest nadajnik - po prostu podajesz nazwę stacji. Radio pamięta czy ktoś słucha - wyłącza się samo gdy nikt nie słucha (auto dispose).

1.2. Konfiguracja Riverpod

pubspec.yaml - Riverpod
pubspec.yaml - zastąp lub dodaj obok provider:
dependencies:
flutter_riverpod: ^2.5.1 # Riverpod dla Flutter (z integracją widgetów)
riverpod_annotation: ^2.3.5 # Opcjonalnie - generowanie kodu przez @riverpod
dev_dependencies:
riverpod_generator: ^2.4.0 # Generuje providery z adnotacji (jak KSP w Androidzie)
build_runner: ^2.4.9 # Runner dla generatora
Uruchom po dodaniu:
flutter pub get
Dla generatora kodu (riverpod_generator) - opcjonalne, ale wygodne:
dart run build_runner watch --delete-conflicting-outputs

main.dart - konfiguracja ProviderScope

// main.dart - owiń całą aplikację w ProviderScope
void main() {
runApp(
const ProviderScope(// WYMAGANE - rejestr wszystkich providerów Riverpoda
child: MyApp(), // Bez ProviderScope: wyjątek MissingProviderScopeException
),
);
}
// MyApp.dart - użyj ConsumerWidget zamiast StatelessWidget
class MyApp extends ConsumerWidget { // <- ConsumerWidget, nie StatelessWidget
const MyApp({super.key});
@override
Widget build(BuildContext context, WidgetRef ref) { // <- dodany WidgetRef ref
// ref = dostęp do providerów (jak context w Provider, ale bezpieczniejszy)
return MaterialApp.router(routerConfig: appRouter, ...);
}
}

2. Typy providerów - kiedy którego użyć?

Riverpod oferuje kilka wyspecjalizowanych typów providerów. Wybór właściwego jest kluczowy - błędny typ to nadmiarowy kod i trudne debugowanie. Poniżej: kompletna mapa decyzji.

Typ providera	Kiedy używać / Co przechowuje / Przykład
Provider<T>	Wartości tylko do odczytu - stałe, konfiguracja, obiekty serwisów. Nie zmienia się w czasie. Przykład: <code>Provider<WeatherService>((ref) => WeatherService())</code> .
StateProvider<T>	Prosty stan mutable - licznik, przełącznik, wybrany indeks. Nie używaj dla złożonych obiektów. Przykład: <code>StateProvider<String>((ref) => 'Warszawa')</code> .
FutureProvider<T>	Dane asynchroniczne tylko do odczytu - jednorazowe żądanie API. Automatycznie zarządza AsyncValue (loading/data/error). Przykład: <code>FutureProvider<WeatherData>((ref) => service.fetch())</code> .
StreamProvider<T>	Dane asynchroniczne ze strumienia - WebSocket, Firestore real-time, sensor. Jak FutureProvider ale dla Stream<T>. Przykład: <code>StreamProvider<double>((ref) => locationStream())</code> .
NotifierProvider<N, T>	Złożony mutable stan z logiką. Klasa Notifier ma metody zmieniające stan. Synchroniczny odpowiednik ChangeNotifier. Przykład: lista zadań add/remove/toggle.
AsyncNotifierProvider<N, T>	Jak NotifierProvider ale dla danych asynchronicznych. Build() zwraca Future<T>. Najlepszy do zarządzania danymi z API z możliwością odświeżania. Przykład: WeatherNotifier z metodą reload().

Dlaczego tyle typów? Czy nie wystarczyłby jeden?

Każdy typ jest zoptymalizowany dla swojego przypadku użycia.
FutureProvider<T> automatycznie anuluje poprzednie żądanie gdy parametr się zmieni,
Provider<Future<T>> tego nie robi.

StateProvider<T> dla prostego inta jest szybszy niż NotifierProvider; mniej boilerplate. Typy są jak narzędzia: młotek, śrubokręt, klucz. Każde robi jedno dobrze, zamiast jedno robi wszystko średnio.

2.1. NotifierProvider - przykład: ulubione miasta

favorites_provider.dart - NotifierProvider	Wyjaśnienie
// Klasa Notifier - logika i stan	Notifier<T> = klasa przechowująca stan i metody go zmieniające
class FavoritesNotifier extends Notifier<List<String>> {	Generyk T = typ stanu. Tu: Lista stringów (nazwy miast)
@override	
List<String> build() {	build() = tworzy stan początkowy. Wywoływane raz przy inicjalizacji.
return ['Warszawa', 'Kraków']; // stan domyślny	Możesz tu async ładować z Hive/SharedPreferences
}	
void addCity(String city) {	Metoda publiczna - widgety wywołują ją przez ref.read
if (state.contains(city)) return; // guard	Sprawdź duplikat przed dodaniem
state = [...state, city]; // NIEZMIENNOŚĆ!	state = nowa lista (nie mutuj istniejącej)! Riverpod powiadomi widgety.
}	
void removeCity(String city) {	
state = state.where((c) => c != city).toList();	Filtruj przez where() zamiast remove() - zwraca nową listę
}	
bool contains(String city) => state.contains(city);	Metoda pomocnicza - getter bez zmiany stanu
}	
// Definicja providera (globalna, poza klasą)	Provider definiowany globalnie - dostępny z każdego miejsca
final favoritesProvider =	final = przypisz raz (stały globalny)
NotifierProvider<FavoritesNotifier, List<String>>(FavoritesNotifier.new,	Generyki: <KlasaNotifier, TypStanu>
);	Przełącz konstruktor klasy (nie instancję)

2.2. AsyncNotifierProvider - dane z API

weather_provider.dart - AsyncNotifierProvider	Wyjaśnienie
class WeatherNotifier	AsyncNotifier<T> = Notifier z asynchronicznym build()
extends AsyncNotifier<WeatherData?> {	T = WeatherData? (nullable - przed załadowaniem)
@override	

weather_provider.dart - AsyncNotifierProvider	Wyjaśnienie
Future<WeatherData?> build() async {	build() jest async - zwraca Future. Riverpod auto-obsługuje loading/error.
return null; // stan początkowy: brak danych	Null = nic nie załadowane. UI zobaczy AsyncValue.data(null).
}	
Future<void> fetch(String city) async {	Publiczna metoda - widgety wywołują ref.read(wp.notifier).fetch(city)
state = const AsyncLoading(); // pokaż spinner	AsyncLoading() = stan ładowania. UI zobaczy CircularProgressIndicator.
state = await AsyncValue.guard(() async {	AsyncValue.guard = try-catch który konwertuje wyjątek na AsyncError
final svc = ref.read(weatherServiceProvider);	ref.read() - odczytaj inny provider (wstrzyknięcie zależności!)
return svc.fetchWeather(city);	Wywołaj serwis. Jeśli rzuci wyjątek, state = AsyncError automatycznie.
});	
}	
Future<void> reload() async {	Odśwież dane - wywołaj ref.invalidateSelf() lub re-fetch
final city = /* zapamiętaj ostatnie miasto */	Przechowaj ostatnie miasto w polu klasy
_lastCity;	
if (city != null) await fetch(city);	
}	
}	
final weatherProvider =	
AsyncNotifierProvider<WeatherNotifier,	
WeatherData?>(() =>	
WeatherNotifier());	() => WeatherNotifier() - anonimowa fabryka (nie .new bo ma zależności)

2.3 Odczyt i modyfikacja stanu w widgetach

ConsumerWidget i ConsumerStatefulWidget
// ConsumerWidget - stateless widget z dostępem do ref
class HomeScreen extends ConsumerWidget {
const HomeScreen({super.key});
@override
Widget build(BuildContext context, WidgetRef ref) {
// ref.watch() = obserwuj i przebuduj przy zmianie (jak context.watch<T>() w Provider)
final favorites = ref.watch(favoritesProvider); // List<String>
final weatherAsync = ref.watch(weatherProvider); // AsyncValue<WeatherData?>
return Scaffold(
body: weatherAsync.when(
// AsyncValue.when() = elegancki switch na wszystkie 3 stany
loading: () => const CircularProgressIndicator(),

error: (err, stack) => ErrorWidget(message: err.toString()),
data: (weather) => WeatherCard(weather: weather),
),
floatingActionButton: FloatingActionButton(
// ref.read() = jednorazowy odczyt / akcja (nie subskrybuje zmian)
onPressed: () => ref.read(weatherProvider.notifier).fetch('Kraków'),
child: const Icon(Icons.refresh),
),
);
}
}
// ConsumerStatefulWidget - stateful widget z ref
class SearchScreen extends ConsumerStatefulWidget {
const SearchScreen({super.key});
@override
ConsumerState<SearchScreen> createState() => _SearchScreenState();
}
class _SearchScreenState extends ConsumerState<SearchScreen> {
// ref jest dostępne bezpośrednio (jak this.ref)
void _onSearch(String city) {
ref.read(weatherProvider.notifier).fetch(city);
ref.read(favoritesProvider.notifier).addCity(city);
}
// ...
}

ref.watch() tylko w build() - nie w callbackach i initState()!

Zasada identyczna jak z context.watch() w Provider: ref.watch() w onPressed() / initState() / didChangeDependencies() → wyjątek ProviderException. ref.read() = wywołaj w callbackach i initState() (jednorazowe działanie). ref.listen() = obserwuj i reaguj na zmiany bez przebudowywania widgetu (np. wyświetl Snackbar).

3. Hive - lokalna baza danych NoSQL

W Flutter Lab 1 wspomniano SharedPreferences do przechowywania prostych wartości (stringów, liczb). SharedPreferences jest odpowiedni dla ustawień. Do przechowywania złożonych obiektów (listy, zagnieżdżone dane, modele) znacznie lepszy jest Hive - lekka, wydajna baza NoSQL napisana w czystym Darcie.

3.1. Hive kontra inne opcje przechowywania

Rozwiązanie	Kiedy używać / Wady i zalety
SharedPreferences	Proste wartości: bool, int, String, List<String>. Synchroniczne API. Nie nadaje się do złożonych obiektów ani dużych danych.
Hive	Obiekty Dart jako rekordy. Bardzo szybki (zapisy w mikros). Brak SQL - proste klucz→wartość i listy. Dobry do offline cache, ulubionych, historii.
Sqflite (SQLite)	Relacyjna baza danych. Dobra gdy potrzebne zapytania SQL, relacje, złożone filtry. Trudniejsze API niż Hive. Odpowiednik Room z Androida.

Rozwiązanie	Kiedy używać / Wady i zalety
drift (dawniej moor)	Type-safe Dart wrapper na SQLite z generowaniem kodu. Jak Room dla Fluttera - bardziej ergonomiczny niż sqflite, ale więcej boilerplate.
Isar	Następca Hive od tego samego autora. Szybszy, z obsługą indeksów i zapytań. Wymaga więcej konfiguracji. Dobry wybór dla nowych projektów zamiast Hive.

3.2 Konfiguracja i inicjalizacja Hive

pubspec.yaml - Hive	
# pubspec.yaml	
dependencies:	
hive: ^2.2.3	# Silnik Hive
hive_flutter: ^1.1.0	# Integracja z Flutter (Hive.initFlutter)
path_provider: ^2.1.3	# Ścieżka do katalogu danych aplikacji
dev_dependencies:	
hive_generator: ^2.0.1	# Generator TypeAdapter z @HiveType/@HiveField
build_runner: ^2.4.9	# Runner generatora (już dodany dla Riverpoda)

main.dart - inicjalizacja Hive
// main.dart - inicjalizacja Hive PRZED runApp()
Future<void> main() async {
// Wymagane przed Hive.initFlutter() i runApp()
WidgetsFlutterBinding.ensureInitialized();
// Inicjalizuj Hive z katalogiem danych aplikacji (DocumentsDirectory na iOS,
// AppData na Androidzie) - hive_flutter robi to automatycznie
await Hive.initFlutter();
// Zarejestruj adaptory dla niestandardowych typów (PRZED openBox!)
Hive.registerAdapter(WeatherDataAdapter()); // generowany przez build_runner
Hive.registerAdapter(CityGeoAdapter());
// Otwórz skrzynki (bazy) - nazwa = identyfikator, jak nazwa tabeli
await Hive.openBox<WeatherData>('weatherCache'); // cache pogody
await Hive.openBox<String>('favorites'); // ulubione miasta (String)
runApp(const ProviderScope(child: MyApp()));
}

3.3. Model danych z adnotacjami Hive

weather_data.dart - model z @HiveType	Wyjaśnienie
import 'package:hive/hive.dart';	Import adnotacji Hive
part 'weather_data.g.dart'; // GENEROWANY plik	part = część tego pliku (generowana przez build_runner)
@HiveType(typeId: 0) // typeId musi być unikalny!	typeId 0-223: unikalne ID dla każdej klasy. Nie zmieniaj po wdrożeniu!

weather_data.dart - model z @HiveType	Wyjaśnienie
<code>class WeatherData extends HiveObject { // HiveObject!</code>	HiveObject = metody .save() i .delete() na obiekcie
<code>@HiveField(0) // indeks pola</code>	HiveField(index): unikalne pole w klasie. Nie zmieniaj kolejności!
<code>late String city;</code>	late = inicjalizacja przez Hive przy odczycie (nie w konstruktorze)
<code>@HiveField(1)</code>	
<code>late double temperature;</code>	
<code>@HiveField(2)</code>	
<code>late int humidity;</code>	
<code>@HiveField(3)</code>	
<code>late DateTime updatedAt;</code>	DateTime jest serializowany natywnie przez Hive
<code>// Konstruktor domyślny (wymagany przez Hive)</code>	Hive tworzy obiekty przez konstruktor domyślny (bezparametrowy)
<code>WeatherData();</code>	
<code>// Konstruktor fabryczny dla wygody</code>	
<code>factory WeatherData.from({</code>	
<code> required String city,</code>	
<code> required double temperature,</code>	
<code> required int humidity,</code>	
<code>}) => WeatherData()</code>	
<code> ..city = city</code>	..city = cascade - przypisz pole i zwróć obiekt
<code> ..temperature = temperature</code>	
<code> ..humidity = humidity</code>	
<code> ..updatedAt = DateTime.now();</code>	
<code>}</code>	

Generowanie kodu Hive
Generowanie kodu adaptera (uruchom w terminalu):
<code>dart run build_runner build --delete-conflicting-outputs</code>
Lub w trybie watch (auto-generuje przy każdej zmianie):
<code>dart run build_runner watch --delete-conflicting-outputs</code>
Efekt: plik weather_data.g.dart z klasą WeatherDataAdapter
Nie edytuj weather_data.g.dart - jest nadpisywany przy każdym build!

3.4. Operacje CRUD na skrzynce Hive

weather_cache_service.dart - operacje Hive	Wyjaśnienie
<code>class WeatherCacheService {</code>	Serwis enkapsulujący operacje na Hive
<code> // Pobierz otwarty Box (już otwarty w main())</code>	Hive.box() - synchroniczny, Box musi być już otwarty
<code> Box<WeatherData> get _box =></code>	

weather_cache_service.dart - operacje Hive	Wyjaśnienie
Hive.box<WeatherData>('weatherCache');	Nie otwieraj Box w serwisie - otwieranie jest async, rób to w main()
<pre>// ZAPIS - klucz: nazwa miasta Future<void> cacheWeather(WeatherData data) async { await _box.put(data.city.toLowerCase(), data); }</pre>	async mimo że Hive jest synchroniczny - dla spójności API put(key, value) - zapisz. Klucz: lowercase nazwa miasta
<pre>// ODCZYT - zwróć null jeśli brak w cache WeatherData? getCache(String city) { return _box.get(city.toLowerCase()); }</pre>	Synchroniczny odczyt - Hive działa w pamięci get(key) - zwróć wartość lub null
<pre>// SPRAWDŹ AKTUALNOŚĆ - starsze niż 30 minut = nieaktualne bool isFresh(String city) { final cached = getCache(city); if (cached == null) return false; final age = DateTime.now().difference(cached.updatedAt); return age.inMinutes < 30; }</pre>	difference() = Duration między dwoma DateTime Dane aktualne przez 30 minut
<pre>// USUŃ jeden wpis Future<void> remove(String city) async { await _box.delete(city.toLowerCase()); }</pre>	delete(key) - usuń klucz
<pre>// LISTA WSZYSTKICH KLUCZY (nazwy miast w cache) List<String> get cachedCities => _box.keys.cast<String>().toList(); }</pre>	keys - Iterable<dynamic>, cast<String> - typed

4. Animacje niejawne; Animated* widgety

Flutter dzieli animacje na dwie kategorie. Animacje niejawne (implicit) obsługują proste przejścia automatycznie - wystarczy zmienić wartość a Flutter sam animuje przejście. Animacje jawne (explicit) dają pełną kontrolę nad czasem, przebiegiem i sekwencją. Zaczniemy od prostszych - niejawnych.

Analogia: automatyczna skrzynia biegów vs. manualna

ANIMACJE NIEJAWNE (Animated*) = automatyczna skrzynia biegów: Podajesz docelową wartość (temperatura, pozycja, kolor). Auto płynnie przejdzie do celu. Idealny dla prostych stanów: pokaż/ukryj, powiększ/pomniejsz, zmień kolor.

ANIMACJE JAWNE (AnimationController) = manualna skrzynia biegów: Kontrolujesz każdą milisekundę. Możesz cofnąć, zatrzymać, zapętlić, synchronizować. Potrzebne gdy animacja jest złożona, zależy od gestu użytkownika lub synchronizuje wiele elementów.

4.1. Przegląd Animated* widgetów

Widget	Co animuje i przykład użycia
AnimatedContainer	Dowolna właściwość Containera: kolor, rozmiar, padding, margin, borderRadius. Najczęściej używany.
AnimatedOpacity	Przezroczystość (0.0-1.0). Efekt pokaż/ukryj. Zamień Opacity → AnimatedOpacity.
AnimatedPositioned	Pozycja w Stack (top, left, right, bottom). Animuje ruch elementu w obrębie Stack.
AnimatedSwitcher	Animowane przejście między RÓŻNYMI widgetami. Automatyczne fade/scale gdy child się zmieni.
AnimatedCrossFade	Płynne przejście między DWOMA konkretnymi widgetami (firstChild ↔ secondChild).
AnimatedList	Animowane dodawanie i usuwanie elementów listy (slide + fade). Wymaga GlobalKey.
AnimatedAlign	Animuje wyrównanie widgetu (Alignment) wewnątrz rodzica.
TweenAnimationBuilder	Animuje dowolną wartość (int, double, Color, Offset) z własnym Tween. Wszystko co nie ma dedykowanego Animated* widgetu.

4.2. AnimatedContainer - płynne przejście właściwości

AnimatedContainer; karta pogody reagująca na stan	Wyjaśnienie
<code>class WeatherCard extends StatefulWidget { ... }</code>	StatefulWidget - stan kontroluje animację
<code>// W _WeatherCardState:</code>	
<code>bool _isExpanded = false; // stan karty</code>	Stan lokalny - rozwinięta/zwinięta karta
<code>@override Widget build(BuildContext context) {</code>	
<code> return GestureDetector(</code>	GestureDetector - detekcja dotyku bez ripple
<code> onTap: () => setState(() => _isExpanded = !_isExpanded),</code>	Zmień stan po tapnięciu → AnimatedContainer się uruchomi
<code> child: AnimatedContainer(</code>	Zamiast Container → AnimatedContainer
<code> duration: const Duration(milliseconds: 300),</code>	Czas trwania animacji - 200-400ms to 'naturalne' przejście
<code> curve: Curves.easeInOut,</code>	Krzywa animacji: easeInOut = powoli start, powoli koniec
<code> // Właściwości animowane - zmieniają się płynnie:</code>	Wszystkie właściwości będą interpolowane przez duration
<code> height: _isExpanded ? 200.0 : 80.0,</code>	Wysokość: 80 → 200 dp (animacja rozwinięcia)
<code> color: _isExpanded</code>	
<code> ?</code>	
<code>Theme.of(context).colorScheme.primaryContainer</code>	Kolor tła zmienia się wraz z rozwinięciem

AnimatedContainer; karta pogody reagująca na stan	Wyjaśnienie
<code>: Theme.of(context).colorScheme.surface,</code>	
<code>padding: EdgeInsets.all(_isExpanded ? 20.0 :</code>	Padding też animowany!
<code>8.0),</code>	
<code>decoration: BoxDecoration(</code>	
<code>borderRadius: BorderRadius.circular(</code>	
<code>_isExpanded ? 24.0 : 12.0),</code>	Zaokrąglenie narożników animowane
<code>),</code>	
<code>child: WeatherInfo(isExpanded: _isExpanded),</code>	Zawartość - nie animowana (tylko kontener)
<code>),</code>	
<code>);</code>	
<code>}</code>	

4.3. AnimatedSwitcher - przejście między widgetami

AnimatedSwitcher - zmiana ikony pogody	Wyjaśnienie
<code>AnimatedSwitcher(</code>	AnimatedSwitcher animuje gdy child się zmienia
<code>duration: const Duration(milliseconds: 400),</code>	
<code>// Customowy przechód: fade + scale (zamiast domyślnego fade)</code>	
<code>transitionBuilder: (child, animation) {</code>	transitionBuilder = jak animujesz stary i nowy child
<code>return ScaleTransition(</code>	ScaleTransition = animacja skali
<code>scale: animation,</code>	animation: 0.0→1.0 dla nowego, 1.0→0.0 dla starego
<code>child: FadeTransition(opacity: animation,</code>	Combine scale + fade dla efektu 'popping in'
<code>child: child),</code>	
<code>);</code>	
<code>},</code>	
<code>// KLUCZOWE: każdy child musi mieć unikalny Key!</code>	Bez Key Flutter myśli że to ten sam widget → brak animacji!
<code>child: Icon(</code>	
<code>_getWeatherIcon(weatherCode), // zmienia się</code>	Gdy weatherCode się zmieni, zmieni się icon - AnimatedSwitcher to zauważy
<code>key: ValueKey<int>(weatherCode), // unikalny Key!</code>	ValueKey(wartość) - identyfikuje unikalnie child dla AnimatedSwitcher
<code>size: 64,</code>	
<code>),</code>	
<code>)</code>	

AnimatedSwitcher bez Key - najczęstszy błąd z animacjami!

Gdy child zmienia się ale Key NIE jest ustawiony, Flutter nie 'widzi' zmiany. Myśli że to ten sam widget o zmienionych właściwościach → brak animacji przejścia.
ZASADA: Każdy child AnimatedSwitcher MUSI mieć unikalny Key.
 Dobre Keys: ValueKey(iconCode), ValueKey(weatherCode), ObjectKey(weatherData).
 Złe Keys: UniqueKey() - tworzy nowy Key przy każdym rebuild → zbyt częste animacje.

5. Animacje jawne - AnimationController

Gdy Animated* widżety nie wystarczają - potrzebujesz pełnej kontroli nad animacją, możliwości jej zatrzymania, cofnięcia lub synchronizacji z gestem - sięgamy po AnimationController. To niskopoziomowe API, ale daje nieograniczone możliwości.

5.1. AnimationController

Pulsujący wskaźnik temperatury z AnimationController	Wyjaśnienie
<code>class PulsingTempIndicator extends StatefulWidget { ... }</code>	StatefulWidget wymagany - AnimationController to mutable state
<code>class _PulsingTempState extends State<PulsingTempIndicator> with SingleTickerProviderStateMixin {</code>	
<code>late AnimationController _controller;</code>	with SingleTickerProviderStateMixin - dostarcza vsync (takt animacji)
<code>late Animation<double> _scaleAnimation;</code>	vsync synchronizuje animację z odświeżaniem ekranu (60fps)
<code>@override void initState() {</code>	late = inicjalizacja w initState (nie w konstruktorze)
<code>super.initState();</code>	Animation<T> = wartości w czasie (Tween zdefiniuje zakres)
<code>_controller = AnimationController(</code>	
<code>vsync: this, // provider</code>	Twórz kontroler ZAWSZE w initState()
<code>taktu (this = mixin)</code>	this = SingleTickerProviderStateMixin dostarcza vsync
<code>duration: const Duration(seconds: 1),</code>	Czas jednego cyklu animacji (0→1)
<code>);</code>	
<code>_scaleAnimation = Tween<double>(</code>	Tween = interpolator: od begin do end
<code>begin: 1.0, // rozmiar normalny</code>	begin = wartość na początku animacji (controller=0)
<code>end: 1.2, // 20% większy</code>	end = wartość na końcu animacji (controller = 1)
<code>).animate(CurvedAnimation(</code>	animate() = połącz Tween z kontrolerem i krzywą
<code>parent: _controller,</code>	parent = skąd bierzemy czas (0.0-1.0)
<code>curve: Curves.easeInOut,</code>	Krzywa kształtuje tempo: easeInOut = naturalne pulse
<code>));</code>	
<code>// Zapętl animację (pulsowanie)</code>	
<code>_controller.repeat(reverse: true); //</code>	reverse:true = animuj do końca, potem wstecz (ping-pong)
<code>0→1→0→1→...</code>	
<code>}</code>	
<code>@override void dispose() {</code>	
<code>_controller.dispose(); // KONIECZNE!</code>	Brak dispose() = wyciek pamięci. AnimationController nie jest GC'owany.
<code>super.dispose();</code>	
<code>}</code>	

Pulsujący wskaźnik temperatury z <code>AnimationController</code>	Wyjaśnienie
<pre>@override Widget build(BuildContext context) { return ScaleTransition(scale: _scaleAnimation, child: Text('\${widget.temp}°C', style: const TextStyle(fontSize: 48, fontWeight: FontWeight.bold),),); }</pre>	<code>ScaleTransition</code> nasłuchuje <code>_scaleAnimation</code> automatycznie Nie potrzeba <code>AnimatedBuilder</code>, <code>ScaleTransition</code> jest gotowy <code>widget.temp</code> = dostęp do pól <code>StatefulWidget</code> z <code>State</code>

5.2 Krzywe animacji - Curves

Krzywa	Zachowanie i kiedy stosować
<code>Curves.linear</code>	Stała prędkość przez cały czas. Mechaniczne, nienatyczne. Używaj rzadko - tylko dla pasków postępu.
<code>Curves.easeIn</code>	Powoli startuje, przyspiesza. Dobry dla elementów wychodzących z ekranu (znikanie).
<code>Curves.easeOut</code>	Szybki start, zwalnia na końcu. Dobry dla elementów wchodzących (pojawianie) - naturalne wyhamowanie.
<code>Curves.easeInOut</code>	Powoli start, przyspiesza w środku, zwalnia na końcu. Najbardziej 'naturalny'. Użyj dla większości animacji UI.
<code>Curves.bounceOut</code>	Efekt odbicia na końcu (sprężyna). Dobry dla elementów które 'lądują' na docelowej pozycji.
<code>Curves.elasticOut</code>	Efekt sprężyny - przekracza cel i wraca. Zabawny, do efektów pull-to-refresh, ikonografii.
<code>Curves.decelerate</code>	Szybki start, stopniowe wyhamowanie. Podobny do <code>easeOut</code> ale bardziej agresywny.
<code>CubicBezier(x1,y1,x2,y2)</code>	Własna krzywa Beziera - pełna kontrola. Przydatny gdy projekt ma niestandardowe krzywe (Material 3: <code>Curves.fastOutSlowIn</code>).

5.3 Hero - animacja między ekranami

Hero to specjalny widget który animuje element między dwoma ekranami - element 'przelatuje' ze swojego miejsca na liście do miejsca w widoku szczegółów. To jeden z najbardziej efektownych efektów UI we Flutterze i wymaga minimalnej konfiguracji.

Hero - implementacja w WeatherApp	Wyjaśnienie
<pre>// Na EKRANIE LISTY (HomeScreen) - karta miasta: Hero(tag: 'weather-icon-\${cityName}', // UNIKALNY tag!</pre>	Hero MUSI mieć identyczny tag na obu ekranach Hero opakuje widget który będzie animowany tag: unikalny identyfikator. Użyj danych (nie 'icon!') dla unikalności.

Hero - implementacja w WeatherApp	Wyjaśnienie
<code>child: Image.asset(</code>	child = widget który 'przeleci' do DetailScreen
<code> 'assets/weather_\${weatherCode}.png',</code>	
<code> width: 48, height: 48,</code>	Rozmiar może być inny niż na DetailScreen - Hero to interpoluje!
<code>),</code>	
<code>),</code>	
<code>// Na EKRANIE SZCZEGÓŁÓW (DetailScreen):</code>	Ten sam tag → Flutter wie że to 'ten sam' element
<code>Hero(</code>	
<code> tag: 'weather-icon-\${cityName}', // ten sam tag!</code>	Identyczny tag jak na liście - Flutter zsynchronizuje
<code> child: Image.asset(</code>	
<code> 'assets/weather_\${weatherCode}.png',</code>	
<code> width: 128, height: 128, // większy rozmiar!</code>	Flutter animuje ROZMIAR podczas lotu - od 48x48 do 128x128
<code>),</code>	
<code>),</code>	
<code>// Nawigacja - GoRouter automatycznie obsługuje Hero!</code>	Hero działa ze wszystkimi systemami nawigacji (GoRouter, Navigator)
<code>context.push('/detail/\${cityName}'); // Hero przeleci sam</code>	Wystarczy nawigować normalnie - Hero resztą zajmie się sam

Trzy reguły Hero: unikalność, spójność i prostota

1. UNIKALNY TAG: Dwa Hero z tym samym tagiem na tym samym ekranie = błąd ('There are multiple heroes...'). Użyj danych w tagu: 'weather-icon-\${cityName}' zamiast 'icon'.
2. SPÓJNY CHILD TYPE: Hero na obu ekranach powinien zawierać podobne widgety (Image → Image). Można animować między różnymi typami, ale wygląd może być niespójny.
3. PROSTOTA: Hero najlepiej działa z obrazami i ikonami. Dla złożonych widgetów użyj customTransitionPage w GoRouter.

6. Testy widgetów - flutter_test

Flutter ma wbudowaną bibliotekę flutter_test do testowania widgetów. W przeciwieństwie do testów instrumentowanych Androida (które wymagają emulatora), testy widgetów Fluttera uruchamiają się wyłącznie na JVM - są szybkie i nie wymagają żadnego urządzenia.

Q Jak Flutter testuje widgety bez emulatora?

Flutter używa własnego silnika renderującego (Skia/Impeller). W trybie testowym Flutter uruchamia uproszczoną wersję tego silnika na maszynie deweloperskiej. Widgety są renderowane do bufora w pamięci - testy mogą sprawdzać drzewo semantyczne, wymiary, teksty i interakcje bez wyświetlania czegokolwiek na ekranie. Jest to niezmiernie szybkie - całą „suite” testów widgetów uruchamia się w sekundy.

Porównanie:

Testy jednostkowe Dart: ~1ms per test

Testy widgetów Flutter: ~10-100ms per test (symulowany render)
 Testy instrumentowane Android: ~5-30s per test (emulator)

6.1 Anatomia testu widgetu

weather_card_test.dart - struktura testu	Wyjaśnienie
<code>import 'package:flutter_test/flutter_test.dart';</code>	flutter_test: główna biblioteka testów widgetów (wbudowana)
<code>import 'package:flutter/material.dart';</code>	Material: potrzebne do MaterialApp i widgetów Material
<code>import '../lib/ui/widgets/weather_card.dart';</code>	Importuj testowany widget
<code>void main() {</code>	main() zawiera wszystkie testy - uruchamiany przez flutter test
<code> // group() = logiczna grupa testów (jak describe w Jest)</code>	group() = folder dla powiązanych testów. Opcjonalny ale zalecany.
<code> group('WeatherCard Widget Tests', () {</code>	Pierwszy argument = nazwa grupy (wyświetlana w raporcie)
<code> // Dane testowe (fixture) - używane w wielu testach</code>	
<code> final testWeather = WeatherData(</code>	Przygotuj dane poza testami (DRY)
<code> city: 'Warszawa',</code>	
<code> temperature: 22.5,</code>	
<code> humidity: 65,</code>	
<code> weatherCode: 1,</code>	
<code>);</code>	
<code> // testWidgets() = jeden test widgetu</code>	Jak @Test w JUnit - jeden przypadek testowy
<code> testWidgets('shows city name', (WidgetTester tester) async {</code>	tester: WidgetTester = narzędzie do interakcji z widgetem
<code> // ARRANGE: zbuduj widget w środowisku testowym</code>	
<code> await tester.pumpWidget(</code>	pumpWidget() = wyrenderuj widget w symulowanym środowisku
<code> MaterialApp(</code>	Owiń w MaterialApp - większość widgetów tego wymaga (Theme, Directionality)
<code> home: WeatherCard(weather: testWeather),</code>	Testowany widget z danymi testowymi
<code>),</code>	
<code>);</code>	
<code> // ACT + ASSERT w jednym kroku (dla prostych przypadków)</code>	
<code> expect(find.text('Warszawa'), findsOneWidget);</code>	find.text() = szukaj widgetu z tym tekstem. findsOneWidget = oczekuj dokładnie 1.
<code> });</code>	
<code> testWidgets('displays temperature with unit', (tester) async {</code>	

weather_card_test.dart - struktura testu	Wyjaśnienie
<code>await tester.pumpWidget(MaterialApp(home: WeatherCard(weather: testWeather)));</code>	
<code>// Sprawdź interpolowany string</code> <code>expect(find.text('22.5°C'), findsOneWidget);</code>	Sprawdź dokładny tekst - jeśli format się zmieni test się złamie (dobra rzecz!)
<code>});</code>	
<code>});</code>	
<code>}</code>	

6.2 Finders, Matchers i akcje - API testów widgetów

API	Opis i przykład
<code>find.text('tekst')</code>	Znajdź widget(y) zawierający(e) ten tekst. Domyślnie: dokładne dopasowanie. <code>find.textContaining('War')</code> dla partial.
<code>find.byType(WeatherCard)</code>	Znajdź widget o danym typie. Użyj gdy szukasz własnych widgetów.
<code>find.byKey(Key('weather-card'))</code>	Znajdź po Key. Najbardziej niezawodny selektor - jak <code>testTag</code> w Compose.
<code>find.byIcon(Icons.thermostat)</code>	Znajdź widget Icon z daną ikoną.
<code>find.bySemanticsLabel('Temperatura')</code>	Znajdź po etykiecie semantycznej (jak <code>contentDescription</code> w Androidzie). Testuje dostępność.
<code>findsOneWidget</code>	Matcher: oczekuj dokładnie 1 widget. Najczęstszy.
<code>findsNothing</code>	Matcher: oczekuj że widget NIE istnieje. Dla stanów ukrytych.
<code>findsNWidgets(n)</code>	Matcher: oczekuj dokładnie n widgetów. Dla list.
<code>findsAtLeastNWidgets(1)</code>	Matcher: oczekuj co najmniej n widgetów.
<code>await tester.tap(finder)</code>	Akcja: tapnij znaleziony widget. Jak <code>performClick()</code> w Compose Testing.
<code>await tester.enterText(finder, 'tekst')</code>	Akcja: wpisz tekst w TextField.
<code>await tester.pump()</code>	Obróć jeden klatka animacji / przetworz pending callbacki. Użyj po każdej akcji.
<code>await tester.pumpAndSettle()</code>	Poczekaj aż wszystkie animacje i Futurey się skończą. Jak <code>waitUntilIdle()</code> w Compose.

6.3 Testowanie interakcji i asynchroniczności

search_screen_test.dart - testy interakcji	Wyjaśnienie
<code>testWidgets('search triggers weather load', (tester) async {</code> <code>// Arrange: przygotuj fake provider Riverpoda</code> <code>await tester.pumpWidget(</code>	Test interakcji: wpisz → submit → sprawdź efekt
<code>ProviderScope(</code> <code>overrides: [</code> <code>// Podmień serwis pogody na fake</code>	ProviderScope na szczycie - wymagane dla Riverpoda overrides = podmień providery na testowe wersje

search_screen_test.dart - testy interakcji	Wyjaśnienie
<code>weatherServiceProvider.overrideWithValue(</code>	<code>overrideWithValue()</code> = podaj konkretną instancję
<code> FakeWeatherService(),</code>	<code>FakeWeatherService</code> = implementacja testowa bez HTTP
<code>),</code>	
<code>],</code>	
<code> child: const MaterialApp(home: HomeScreen()),</code>	
<code>),</code>	
<code>);</code>	
<code> // Act: znajdź TextField i wpisz miasto</code>	
<code> await tester.enterText(</code>	<code>enterText</code> = wpisz text w znalezionym TextField
<code> find.byKey(const Key('search-field')),</code>	Szukamy po Key (ustawiony w SearchBar Modifier.key)
<code> 'Gdańsk',</code>	Wpisywany tekst
<code>);</code>	
<code> await</code>	Symuluj wciśnięcie klawisza Search na klawiaturze
<code>tester.testTextInput.receiveAction(TextInputAction.search);</code>	
<code> // pumpAndSettle czeka na Future (ładowanie danych)</code>	
<code> await tester.pumpAndSettle();</code>	Czekaj aż FutureProvider zakończy i UI się zaktualizuje
<code> // Assert: sprawdź że wynik się pojawił</code>	
<code> expect(find.text('Gdańsk'), findsOneWidget);</code>	Nazwa miasta widoczna w wynikach
<code> expect(find.text('Ładowanie...'), findsNothing);</code>	Spinner już zniknął
<code> expect(find.byType(WeatherCard), findsOneWidget);</code>	Karta pogody widoczna
<code>});</code>	

6.4 Uruchamianie testów

flutter test; uruchamianie testów
Uruchom wszystkie testy:
<code>flutter test</code>
Uruchom konkretny plik:
<code>flutter test test/ui/widgets/weather_card_test.dart</code>
Uruchom z verbose output (nazwa każdego testu):
<code>flutter test --reporter expanded</code>
Uruchom z coverage (raport HTML):
<code>flutter test --coverage</code>
<code>genhtml coverage/lcov.info -o coverage/html</code>
<code>open coverage/html/index.html # macOS/Linux</code>
Filtruj po nazwie testu (--name = regex):

flutter test --name 'shows city name'
Obserwuj i uruchamiaj przy zmianach (jak --watch w Jest):
flutter test --watch # wymaga flutter SDK >= 3.13

7. Mocktail - mockowanie zależności

W testach jednostkowych i widgetów często potrzebujemy zastąpić prawdziwe implementacje (serwis HTTP, baza danych) kontrolowanymi obiektami testowymi. Mocktail to biblioteka mockująca dla Dart - odpowiednik MockK z ekosystemu Kotlin.

7.1 Konfiguracja Mocktail

pubspec.yaml - Mocktail
dev_dependencies:
mocktail: ^1.0.4 # Mockowanie dla Dart (lepsze od mockito dla null safety)
Mocktail nie wymaga generowania kodu (w odróżnieniu od mockito).
Działa przez dziedziczenie i manual stubs.

7.2 Tworzenie mocków i stubów

weather_provider_test.dart - Mocktail	Wyjaśnienie
import 'package:mocktail/mocktail.dart';	Import Mocktail
// 1. Zdefiniuj klasę mocka - extends Mock + implementuj interfejs class MockWeatherService extends Mock implements WeatherService {}	Jak class MockRepo extends MockK w MockK (Kotlin) Mock 'pusty' - wszystkie metody domyślnie rzucają wyjątek
// 2. Zarejestruj fallback values dla typów niestandardowych setUpAll(() { registerFallbackValue(FakeWeatherData()); // jeśli potrzebne });	Wymagane gdy Mocktail nie zna typu argumentu setUpAll = raz przed wszystkimi testami (jak @BeforeAll w JUnit 5) Zarejestruj 'pusty' obiekt jako domyślny dla any()
void main() { late MockWeatherService mockService; late WeatherNotifier notifier; setUp(() { mockService = MockWeatherService(); notifier = WeatherNotifier(service: mockService); });	late = inicjalizacja w setUp() setUp = przed każdym testem (jak @BeforeEach w JUnit 5) Nowy mock dla każdego testu - izolacja Wstrzyknij mock przez konstruktor

weather_provider_test.dart - Mocktail	Wyjaśnienie
test('fetch sets loading then data', () async {	test() = test jednostkowy (bez widgetu)
// STUB: zdefiniuj co mock zwróci	when().thenAnswer() = jak coEvery {} returns w MockK
when(() => mockService.fetchWeather('Kraków'))	when() + strzałka = identyfikacja wywoływanej metody
.thenAnswer((_) async => fakeWeatherData);	thenAnswer() dla async. thenReturn() dla sync.
// ACT: wywołaj metodę notifiersa	
await notifier.fetch('Kraków');	
// ASSERT: stan po operacji	
expect(notifier.state,	isA<T>() = sprawdź typ. AsyncData = sukces w Riverpod.
isA<AsyncData<WeatherData?>>());	
// VERIFY: czy metoda została wywołana?	verify() = sprawdź interakcję (jak coVerify w MockK)
verify(() =>	called(1) = wywołana dokładnie 1 raz
mockService.fetchWeather('Kraków')).called(1);	
verifyNever(() =>	verifyNever = nigdy nie wywołana z tym argumentem
mockService.fetchWeather('Gdańsk'));	
});	
}	

8. Architektura projektu - Feature-First i Clean Architecture

W Flutter Lab 1 stosowaliśmy prostą strukturę lib/ z katalogami data/, providers/, ui/. To podejście działa dla małych projektów. Przy rozroście aplikacji lepsza jest architektura Feature-First - każda funkcjonalność ma swój własny, samodzielny moduł.

8.1 Layer-first vs Feature-first

Layer-first (Lab 1)	Feature-first (rekomendowany)
lib/data/, lib/providers/, lib/ui/ grupowanie po typie technicznym. Dobrze dla < 5 ekranów. Problemy: plik weather_provider.dart jest 'sąsiadem' cart_provider.dart mimo że to różne funkcje.	lib/features/weather/, lib/features/favorites/ - grupowanie po funkcji biznesowej. Każda feature jest samowystarczalna: ma własne data, providers, ui.
Zmiana jednej funkcji dotyka wielu katalogów (data/, providers/, ui/). Naukowiec, który czyta kod, musi skakać między katalogami.	Zmiana jednej funkcji odbywa się w jednym miejscu (features/weather/). Naukowiec widzi od razu co należy do 'pogody'.

8.2 Struktura Feature-First dla WeatherApp v2

Struktura Feature-First projektu WeatherApp v2	
lib/	
├─ main.dart	
├─ core/	← Współdzielone przez wszystkie features
└─ error/	

				└─ exceptions.dart	← Własne klasy wyjątków (WeatherException itp.)
				└─ failures.dart	← Sealed class Failure (NetworkFailure, CacheFailure)
				└─ network/	
				└─ api_client.dart	← Wspólny http.Client z interceptorami
				└─ router/	
				└─ app_router.dart	← GoRouter - cała nawigacja
				└─ theme/	
				└─ app_theme.dart	← ThemeData light i dark
				└─ features/	
				└─ weather/	← Feature: aktualna pogoda
				└─ data/	
				└─ models/	
				└─ weather_dto.dart	← DTO: parsowanie JSON z API
				└─ weather_data.dart	← Domain model + Hive @HiveType
				└─ services/	
				└─ weather_api_service.dart	← Wywołania HTTP
				└─ repositories/	
				└─ weather_repository.dart	← łączy API + cache Hive
				└─ providers/	
				└─ weather_provider.dart	← AsyncNotifierProvider
				└─ weather_service_provider.dart	← Provider<WeatherRepository>
				└─ ui/	
				└─ screens/	
				└─ detail_screen.dart	
				└─ widgets/	
				└─ weather_card.dart	
				└─ forecast_tile.dart	
				└─ favorites/	← Feature: ulubione miasta
				└─ data/	
				└─ repositories/	
				└─ favorites_repository.dart	← Hive Box<String>
				└─ providers/	
				└─ favorites_provider.dart	← NotifierProvider
				└─ ui/	
				└─ screens/	
				└─ home_screen.dart	
				└─ widgets/	
				└─ city_tile.dart	
				└─ shared/	← Widgety używane w wielu features
				└─ widgets/	
				└─ loading_widget.dart	
				└─ error_widget.dart	

8.3 Repository Pattern - warstwa między API a UI

weather_repository.dart; offline-first z Hive + API	Wyjaśnienie
class WeatherRepository {	Repository = orchestrator: API + cache. UI nie wie 'skąd' dane.
final WeatherApiService _api;	Wstrzyknięte przez konstruktor (dependency injection)
final WeatherCacheService _cache;	

weather_repository.dart; offline-first z Hive + API	Wyjaśnienie
const WeatherRepository({ required WeatherApiService api, required WeatherCacheService cache, }) : _api = api, _cache = cache;	named parameters - czytelne przy tworzeniu : _api = api = inicjalizacja pola w liście inicjalizacyjnej
Future<WeatherData> getWeather(String city) async { // KROK 1: Sprawdź czy cache jest świeży (< 30 min) if (_cache.isFresh(city)) { return _cache.getCached(city)!; // z cache }	Główna metoda - cache-first Offline-first: najpierw spróbuj lokalnie
// KROK 2: Pobierz z API i zapisz do cache try { final data = await _api.fetchWeather(city); await _cache.cacheWeather(data); return data; } catch (e) {	Sieć: tylko gdy cache nieaktualne Zapisz do Hive - następne wywołanie pobierze z cache
// KROK 3: Błąd sieci - zwróć stary cache jeśli istnieje final stale = _cache.getCached(city); if (stale != null) return stale; rethrow; // Brak cache - propaguj błąd do UI }	Graceful degradation: stare dane lepsze niż błąd getCached() = zwróć nieaktualne dane jeśli są Zwróć nawet stare dane (lepsz UX niż crash) rethrow = rzuć ten sam wyjątek dalej (nie trać stack trace)
}	
}	

9. Dostępność i internacjonalizacja

Profesjonalna aplikacja mobilna musi być dostępna dla użytkowników z niepełnosprawnościami oraz działać w wielu językach. Flutter zapewnia wbudowane mechanizmy dla obu tych wymagań.

9.1 Dostępność - Semantics widget

Flutter generuje drzewo semantyczne automatycznie dla standardowych widgetów (Text, ElevatedButton, TextField). Dla niestandardowych widgetów i gestów musimy je opisać ręcznie przez widget Semantics.

Semantics; dostępność widgetu pogody	Wyjaśnienie
// Karta temperatury z pełną semantyką TalkBack/VoiceOver	

Semantics; dostępność widgetu pogody	Wyjaśnienie
Semantics(label: 'Temperatura w Warszawie: 22.5 stopnia Celsjusza', hint: 'Dwukrotnie stuknij, aby zobaczyć prognozę', button: true, onTap: () => context.push('/detail/Warszawa'), child: Container(child: Row(children: [// excludeSemantics: true = ukryj przed czytnikiem ExcludeSemantics(child: Icon(Icons.thermostat, size: 32),), Text('22.5°C', style: ...),]),),),	Semantics = opisuje widget dla czytnika ekranu label = co czytnik ekranu przeczyta. Pełny opis, nie skrót. hint = instrukcja dla użytkownika. Co się stanie po tapnięciu. button: true = czytnik ogłasza 'przycisk' i pozwala aktywować onTap w Semantics = jak onPressed - dostępny dla assistive tech Owiń widget który nie ma sam semantyki ExcludeSemantics = ikona dekoracyjna, czytnik pomija Ikona jest dekoracyjna - label Semantics już opisuje wszystko Text ma automatyczną semantykę - czytnik przeczyta '22.5°C'

9.2 Internacjonalizacja (i18n); flutter_localizations

pubspec.yaml - i18n
pubspec.yaml
dependencies:
flutter_localizations: # wbudowana w Flutter SDK - bez wersji
sdk: flutter
intl: ^0.19.0 # Formatowanie liczb, dat, walut
flutter:
generate: true # Włącz generator plików .arb → .dart

Pliki ARB - tłumaczenia
lib/l10n/app_pl.arb - polskie tłumaczenia
{
"@@locale": "pl",
"appTitle": "Pogodynka",
"searchHint": "Wpisz nazwę miasta...",
"temperatureLabel": "Temperatura: {temp}°C",
"@temperatureLabel": {
"placeholders": { "temp": { "type": "double", "format": "decimalPattern" } }
},
"humidity": "Wilgotność: {value}%",
"@humidity": {
"placeholders": { "value": { "type": "int" } }
}
}

lib/l10n/app_en.arb - angielskie tłumaczenia
{
"@@locale": "en",
"appTitle": "Weather App",
"searchHint": "Enter city name...",
"temperatureLabel": "Temperature: {temp}°C",
"humidity": "Humidity: {value}%"
}
Generuj klasy Dart z plików .arb:
flutter gen-l10n
Użycie w kodzie (po generacji):
Text(AppLocalizations.of(context)!.appTitle)
Text(AppLocalizations.of(context)!.temperatureLabel(22.5))

MaterialApp - konfiguracja lokalizacji
// main.dart - konfiguracja lokalizacji
MaterialApp.router(// Delegaci lokalizacji - MUSZĄ być ustawione localizationsDelegates: const [AppLocalizations.delegate, // Własne tłumaczenia (generowane) GlobalMaterialLocalizations.delegate, // Material widgets (daty, przyciski OK/Cancel) GlobalWidgetsLocalizations.delegate, // Podstawowe widgety (kierunek tekstu LTR/RTL) GlobalCupertinoLocalizations.delegate, // iOS-style widgets], // Obsługiwane języki supportedLocales: const [Locale('pl', 'PL'), // Polski (Polska) Locale('en', 'US'), // Angielski (USA)], // Domyślna lokalizacja gdy system nie pasuje do żadnej obsługiwanej locale: const Locale('pl', 'PL'), routerConfig: appRouter,)

10. Flutter DevTools - diagnostyka i profilowanie

Flutter DevTools to zaawansowane narzędzie debugowania uruchamiane w przeglądarce. Dostarcza wizualizację drzewa widgetów na żywo, narzędzie do profilowania wydajności, inspektor sieci i debugger pamięci. Znajomość DevTools jest niezbędna przy optymalizacji aplikacji.

#	Uruchomienie Flutter DevTools
1	Uruchom aplikację w trybie debug: flutter run (lub przez Android Studio przycisk Debug).
2	W terminalu pojawi się URL DevTools, np.: 'An Observatory debugger and profiler on...'. Wpisz w przeglądarce lub kliknij link.
3	Alternatywnie: w Android Studio otwórz View → Tool Windows → Flutter Inspector. DevTools otworzy się w panelu bocznym.

#	Uruchomienie Flutter DevTools
4	Możesz też uruchomić DevTools niezależnie: dart devtools - otworzy się serwer, wklej URL aplikacji.

Panel DevTools	Do czego służy
Flutter Inspector	Drzewo widgetów na żywo. Zaznaczaj widgety w aplikacji i inspektor pokaże je w drzewie. Wyświetla rozmiary, padding, margin. Poszukaj 'Debug Paint' - pokaże granice widgetów.
Performance	Timeline renderowania - każda klatka, czas CPU i GPU. Szukaj czerwonych/żółtych klatek (> 16ms). Nagrywaj profile dla konkretnych akcji.
CPU Profiler	Nagrywanie call stack w czasie. Sprawdź co zajmuje czas CPU podczas ładowania danych lub renderowania listy.
Memory	Użycie pamięci w czasie. Wykrywanie wycieków pamięci – obserwuj, czy pamięć rośnie po wielokrotnych akcjach bez powrotu do baseline.
Network	Wszystkie żądania HTTP. Czas odpowiedzi, kody statusu, nagłówki, ciało odpowiedzi. Idealne do debugowania API calls bez Postmana.
Logging	Logi aplikacji z filtrami. Wyświetla print(), debugPrint() i wyjątki. Lepsze niż terminal gdy jest wiele logów.
App Size	Analiza rozmiaru APK/IPA. Pokaże co zajmuje miejsce (assets, paczki, kod). Uruchom przez: flutter build apk --analyze-size.

Repaint Rainbow; znajdź zbędne przebudowania widgetów

W Flutter Inspector włącz 'Highlight Repaints' (przełącznik u góry). Widgety które się przebudowują będą migać kolorowymi ramkami. Częste miganie = zbędny rebuild - optymalizuj przez:

1. const konstruktory na widgetach które się nie zmieniają.
2. Zmiana watch() na select() w Riverpod - subskrybuj tylko część stanu.
3. Wydziel często zmieniający się widget do osobnej klasy (mniejszy zakres rebuilda).

11. Zadania do wykonania

Zadania są kontynuacją projektu WeatherApp z Flutter Lab 1. Kod z Lab 1 służy jako punkt startowy. Migrujemy go do nowej architektury i rozbudowujemy o nowe funkcje.

Zadanie 1 - Migracja na Riverpod i Hive

- 1.1 Dodaj paczki do pubspec.yaml: flutter_riverpod, hive, hive_flutter, hive_generator, build_runner.
- 1.2 Owiń aplikację w ProviderScope w main.dart. Zainicjalizuj Hive przez Hive.initFlutter() przed runApp().
- 1.3 Dodaj adnotacje @HiveType i @HiveField do klasy WeatherData. Uruchom build_runner, zarejestruj wygenerowany adapter w main.dart, otwórz Box<WeatherData>('weatherCache').
- 1.4 Zmigruj WeatherProvider z ChangeNotifier (Provider) na AsyncNotifierProvider (Riverpod). Zmigruj FavoritesProvider na NotifierProvider<FavoritesNotifier, List<String>>.
- 1.5 Zmień HomeScreen i DetailScreen: zastąp Consumer<T> przez ConsumerWidget (WidgetRef ref).

WERYFIKACJA: Aplikacja działa identycznie jak po Lab 1. Brak importów package:provider.

Zadanie 2 - Offline-first z Hive i Repository Pattern

- 2.1 Wdrożenie WeatherCacheService: metody cacheWeather(), getCache(), isFresh() (30 min TTL), remove().
- 2.2 Wdrożenie WeatherRepository łączącego API + cache: cache-first, fallback na stary cache przy błędzie sieci.

Provider<WeatherRepository> zarejestrowany w Riverpodzie.
2.3 Zmodyfikuj WeatherNotifier aby używał WeatherRepository (ref.read(repositoryProvider)) zamiast bezpośrednio serwisu.
2.4 Przetestuj offline-first: załaduj pogodę → wyłącz internet → uruchom ponownie aplikację → dane nadal widoczne.
2.5 Wyświetl w DetailScreen znacznik czasu ostatniej aktualizacji danych (weather.updatedAt - sformatowany przez intl).
WERYFIKACJA: Screenshot z danymi widocznymi przy wyłączonym locie samolotowym. Timestamp widoczny w UI.

Zadanie 3 - Animacje
3.1. AnimatedContainer na karcie miasta w HomeScreen: tapnięcie rozszerza kartę (h: 80→180dp) z efektem animowanego koloru tła. Czas: 350ms, Curves.easeInOut.
3.2. AnimatedSwitcher dla ikony pogody w WeatherCard: zmiana weatherCode animuje ikonę (ScaleTransition + FadeTransition). Pamiętaj o ValueKey(weatherCode).
3.3. Hero między HomeScreen a DetailScreen: ikona pogody (64×64 na liście, 128×128 w szczegółach). Tag: 'weather-icon-\$city'. Sprawdź płynne przejście.
3.4. Animacja wejścia listy w HomeScreen: ListView z animowanym pojawianiem się kart (SlideTransition + FadeTransition). Każda karta przesuwa się od prawej ze staggered delay (karta_n × 50ms opóźnienia).
WERYFIKACJA: Demo animacji na AVD przy 60fps (flutter run --release dla lepszego pomiaru).

Zadanie 4 - Testy widgetów i jednostkowe
4.1. Napisz min. 4 testy jednostkowe (flutter test, bez widgetów): WeatherData.fromJson() dla poprawnych danych, pustego JSON, JSON z nullem w wymaganym polu, przeliczenie temperatury z K na °C.
4.2. Napisz min. 3 testy widgetu WeatherCard (testWidgets): wyświetlanie nazwy miasta, temperatury z jednostką, ikony pogody dla weatherCode == 0 (słonecznie).
4.3. Napisz test interakcji SearchBar: wpisz 'Gdańsk' → wciśnij Search → sprawdź że FakeWeatherService.fetchWeather() wywołane z 'Gdańsk'.
Użyj Mocktail do mockowania WeatherService i ProviderScope(overrides:[])
do wstrzyknięcia mocka.
4.4. Uruchom flutter test -coverage. Uzyskaj pokrycie > 60% dla pliku weather_card.dart.
WERYFIKACJA: flutter test uruchamia się bez błędów. Raport coverage w coverage/html/index.html.

12. Kryteria oceniania

Zadanie	Punkty	Co weryfikuje prowadzący
Zad. 1: Riverpod + Hive	20 pkt	Brak package:provider. ProviderScope. Hive inicjalizacja + adapter. AsyncNotifierProvider i NotifierProvider działają.
Zad. 2: Offline-first	25 pkt	Dane widoczne przy wyłączonym internecie. Timestamp w UI. Repository Pattern (API + cache oddzielone).
Zad. 3: Animacje	30 pkt	AnimatedContainer (expand). AnimatedSwitcher z ValueKey. Hero między ekranami. Staggered list entry.
Zad. 4: Testy	25 pkt	Min. 7 testów (4 jednostkowe + 3 widgetów). Mocktail mock. Coverage > 60% dla WeatherCard.
RAZEM	100 pkt	

12.1 Skala ocen

Ocena	Punkty / Wymagania
5.0	90-100 pkt - Wszystkie zadania. Animacje płynne w release. Coverage > 70%. Mocktail z verify().
4.5	80-89 pkt - Zadania 1-3 kompletne. Min. 5 testów. Hero działa. Offline-first.
4.0	70-79 pkt - Zadania 1-2 + AnimatedContainer i AnimatedSwitcher. Min. 3 testy widgetu.
3.5	60-69 pkt - Zadania 1-2. Riverpod i Hive działają. Brak animacji. Min. 4 testy jednostkowe.
3.0	50-59 pkt - Zadanie 1: migracja na Riverpod działa. Dane z Hive persystują.
2.0	0-49 pkt - Projekt nie kompiluje lub Riverpod nie działa.

13. Najczęstsze błędy i ich rozwiązania

Błąd / Objaw	Przyczyna i rozwiązanie
MissingProviderScopeException / ProviderNotFoundException	Brak ProviderScope w drzewie widgetów. Owiń całą aplikację: runApp(ProviderScope(child: MyApp())).
HiveError: 'Box not found. Did you forget to call Hive.openBox?'	Box nie został otwarty przed użyciem. Hive.openBox() MUSI być wywołane w main() przed runApp(). Nie otwieraj Box w konstruktorze klasy serwisu.
Bad state: already registered typeId 0	Dwa adaptory mają ten sam typeld. Każda klasa @HiveType musi mieć unikalny typeld (0, 1, 2...). Sprawdź wszystkie @HiveType w projekcie.
AnimatedSwitcher nie animuje przy zmianie child	Brak unikalnego Key w child. Dodaj ValueKey(unikalnaWartość) do każdego child AnimatedSwitcher.
Hero: 'There are multiple heroes that share the same tag'	Dwa widgety Hero z tym samym tagiem na tym samym ekranie. Użyj unikalnych tagów: 'weather-icon-\$cityName' zamiast 'weather-icon'.
AnimationController nie zwalnia pamięci (wyciek)	Brak _controller.dispose() w metodzie dispose() State. Zawsze: @override void dispose() { _controller.dispose(); super.dispose(); }
ref.watch() throws ProviderException poza build()	ref.watch() wywołane w initState(), onPressed() lub async callback. Zamień na ref.read() dla akcji poza build().
Mocktail: MissingStubError - no stub for method	Metoda mocka wywołana bez wcześniejszego when(). Dodaj when(() => mock.method()).thenAnswer((_) async => result) przed wywołaniem kodu testowanego.
flutter test - Cannot find widget with type X	Widget nie jest wyrenderowany w testowym drzewie. Sprawdź czy pumpWidget() zawiera MaterialApp jako korzeń i czy widget jest w strukturze children.
Hive: 'Cannot write to a closed box'	Box zamknięty przed zapisem lub operacja po dispose(). Upewnij się że Box jest otwarty przez cały czas życia aplikacji.

Debugging Riverpod - ProviderObserver

Riverpod umożliwia obserwowanie wszystkich zmian stanu przez ProviderObserver. Dodaj do ProviderScope(observers: [LoggingObserver()],) gdzie:

```
class LoggingObserver extends ProviderObserver {
  @override
  void didUpdateProvider(provider, prev, next, container) {
    debugPrint('[Riverpod] ${provider.name}: $prev → $next');
  }
}
```

Instrukcja Laboratoryjna Flutter 2 - Riverpod, Animacje, Hive i Testy Widgetów
 Programowanie Aplikacji Mobilnych | Katedra Informatyki i Automatyki
MobileHub

Następne ćwiczenie: Flutter 3 - Publikacja aplikacji, CI/CD z GitHub Actions, Flavors

